

Institute of Engineering & Management, Kolkata

Department of Computer Science and Engineering

Python API Automation Framework

Requests + Behave BDD + Allure Reporting

Capstone Project Report

Selenium / API Testing Coursework

Submitted by:

Shivraj Bhattacharjee

Mentor:

Prof. Sramana Mukherjee

September 26, 2026

Contents

1	Introduction	2
1.1	Objective	2
1.2	Scope	2
1.3	Tools and Technologies Used	2
2	System Design and Architecture	3
2.1	Layered Architecture	3
2.2	Project Folder Structure	3
3	Implementation Details	5
3.1	Configuration Layer	5
3.2	Base API Client	5
3.3	Endpoint-Specific Clients	5
3.4	JSON Schema Validation	6
3.5	BDD Test Scenarios	6
3.6	Allure Reporting	6
4	Testing and Results	7
4.1	Test Execution	7
4.2	Dry-Run Verification	7
4.3	Test Scenarios Covered	7
4.4	Screenshots	7
4.4.1	Console Output — Behave Test Run	7
4.4.2	Allure Report — Overview Dashboard	7
5	Conclusion	9
5.1	Summary	9
5.2	Future Enhancements	9
5.3	Repository	9

1. Introduction

1.1 Objective

The objective of this project is to design and implement a complete **API Test Automation Framework** using Python. The framework demonstrates industry-standard practices for testing REST APIs, including:

- Sending and validating HTTP requests using the `requests` library
- Behavior-Driven Development (BDD) test design using `Behave`
- Structured, human-readable test reporting using `Allure`
- Authentication and token-based session handling
- JSON Schema (contract) validation of API responses
- Reusable, modular framework architecture

1.2 Scope

The framework targets the User Management API hosted at <https://jsonplaceholder.typicode.com/>, a free fake REST API used for testing and prototyping, along with an authentication demo against <https://automationexercise.com/api>.

1.3 Tools and Technologies Used

Tool / Library	Purpose
Python 3.x	Core programming language
Requests	HTTP client for calling REST endpoints
Behave	BDD framework for Gherkin-style test scenarios
Allure (allure-behave)	Rich, interactive HTML test reports
jjsonschema	Contract/schema validation of API responses
PyYAML	Loading YAML-based test data

Table 1.1: Technology stack used in the framework

2. System Design and Architecture

2.1 Layered Architecture

The framework follows a four-layer design so that responsibilities are cleanly separated and the codebase stays maintainable and reusable:

1. **Feature Layer** — Gherkin `.feature` files describing test scenarios in plain English.
2. **Step Definition Layer** — Python glue code that maps Gherkin steps to framework calls.
3. **API Client Layer** — Reusable classes wrapping the `requests` library for each API resource.
4. **Utility Layer** — Shared helpers for logging, assertions, schema validation, and data loading.

```
Feature Files (Gherkin)
    |
    v
Step Definitions (Python)
    |
    v
API Client Layer (Requests wrapper)
    |
    v
REST API (jsonplaceholder.typicode.com)
```

2.2 Project Folder Structure

```
1 api-automation-framework/
2 |-- features/
3 |   |-- user_management.feature
4 |   |-- auth.feature
5 |   '-- steps/
6 |       |-- user_steps.py
7 |       '-- auth_steps.py
8 |-- api_client/
9 |   |-- base_client.py
10 |   |-- user_api.py
11 |   '-- auth_api.py
12 |-- config/
```

```
13 | |-- config.py
14 | '-- endpoints.py
15 |-- schemas/
16 | |-- user_schema.json
17 | |-- user_list_schema.json
18 | |-- created_user_schema.json
19 | '-- error_schema.json
20 |-- test_data/
21 | |-- users.json
22 | '-- test_config.yaml
23 |-- utils/
24 | |-- logger.py
25 | |-- assertions.py
26 | |-- schema_validator.py
27 | '-- data_reader.py
28 |-- environment.py
29 |-- behave.ini
30 |-- requirements.txt
31 '-- reports/allure-results/
```

Listing 2.1: Project directory layout

3. Implementation Details

3.1 Configuration Layer

All environment-specific values (base URL, timeouts, default headers) are centralized in `config/config.py`, allowing the framework to switch environments via a single environment variable (`API_ENV`) without touching any test code.

3.2 Base API Client

The `BaseAPIClient` class wraps the `requests` library and provides:

- Centralized header and timeout management
- Automatic request/response logging
- Automatic attachment of request/response bodies to Allure reports
- Bearer token support via `set_auth_token()`

```
1 def _request(self, method: str, path: str, **kwargs):
2     url = self._full_url(path)
3     logger.info(f"--> {method} {url}")
4
5     response = self.session.request(
6         method=method, url=url,
7         headers=self.headers, timeout=self.timeout, **kwargs,
8     )
9
10    logger.info(f"<-- {response.status_code} {url}")
11    self._attach_to_allure(method, url, kwargs.get("json"), response)
12    return response
```

Listing 3.1: Excerpt from `base_client.py`

3.3 Endpoint-Specific Clients

`UserAPI` and `AuthAPI` inherit from `BaseAPIClient` and expose resource-specific methods such as `get_user()`, `create_user()`, `update_user()`, `delete_user()`, and `login()`.

3.4 JSON Schema Validation

A key feature of this framework is contract testing — verifying that API responses match an expected structure (required fields, correct data types) rather than only checking status codes. This is handled by `utils/schema_validator.py` using the `jsonschema` library, with schema definitions stored under `schemas/`.

```
1 def validate_schema(response_json, schema_filename):
2     schema = load_schema(schema_filename)
3     try:
4         validate(instance=response_json, schema=schema)
5         return True, None
6     except ValidationError as e:
7         return False, str(e)
```

Listing 3.2: Schema validation function

3.5 BDD Test Scenarios

Test cases are written in Gherkin syntax so they remain readable to non-technical stakeholders.

```
1 Scenario: Create a new user with valid data
2   When I send a POST request to create a user with valid data
3   Then the response status code should be 201
4   And the response should match the "created_user_schema.json" schema
5   And the response field "name" should equal "Shivraj Sharma"
```

Listing 3.3: Sample scenario from user_management.feature

3.6 Allure Reporting

Every request and response is automatically attached to the Allure report as a text artifact, making it easy to debug failing scenarios without re-running tests.

4. Testing and Results

4.1 Test Execution

Tests were executed using the following command:

```
1 behave -f allure_behave.formatter:AllureFormatter -o reports/allure-  
    results
```

4.2 Dry-Run Verification

Before full execution, a dry run was performed to confirm that every Gherkin step correctly binds to a step definition:

```
1 behave --dry-run
```

All 32 steps across both feature files resolved successfully with zero undefined steps.

4.3 Test Scenarios Covered

Scenario	Description	Tag
Get all users	Validates list endpoint + schema	@smoke
Get user by ID	Validates single user schema	@get
Get non-existent user	Expects 404	@negative
Create user	Validates 201 + created schema	@post
Update user	Validates field update	@put
Delete user	Validates deletion response	@delete
Login (valid)	Validates successful auth	@auth
Login (invalid)	Validates failure message	@negative

Table 4.1: Summary of test scenarios

4.4 Screenshots

4.4.1 Console Output — Behave Test Run

4.4.2 Allure Report — Overview Dashboard

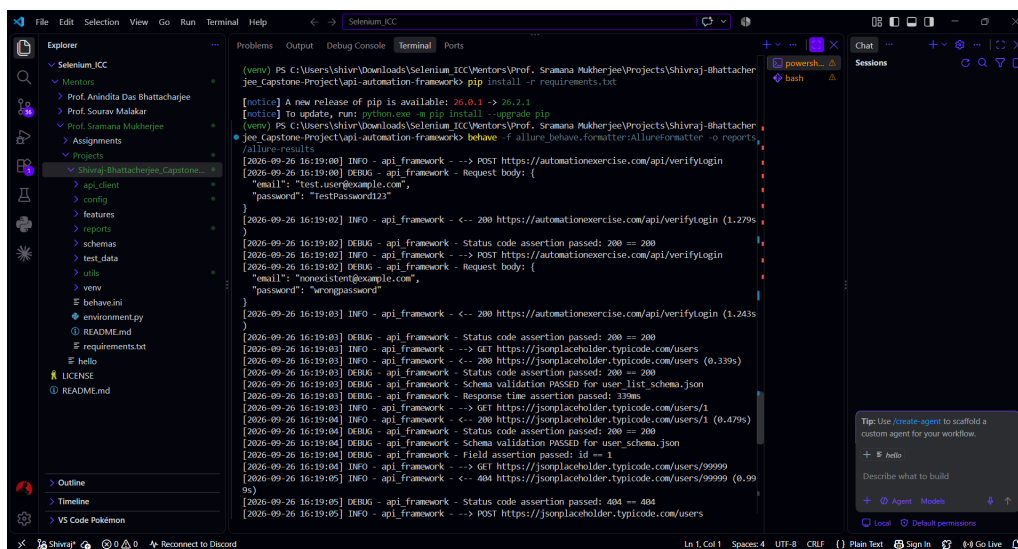


Figure 4.1: Behave console output showing scenario execution

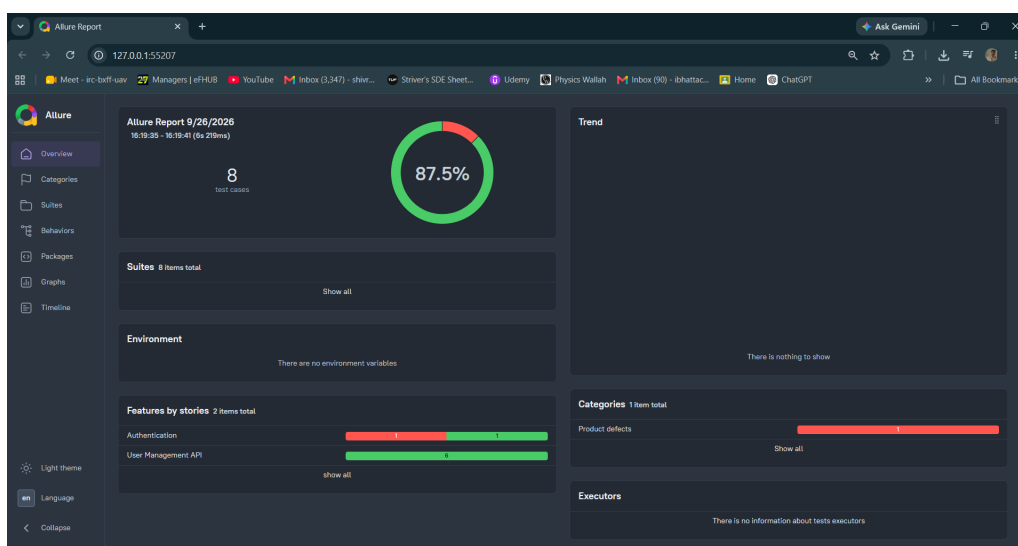


Figure 4.2: Allure report overview dashboard

5. Conclusion

5.1 Summary

This project successfully implements a reusable, layered API test automation framework combining `Requests`, `Behave`, and `Allure`. It demonstrates CRUD operation testing, authentication handling, and JSON schema-based contract validation — going beyond simple status-code checks to verify full response correctness.

5.2 Future Enhancements

- Integrate with a CI/CD pipeline (GitHub Actions) for automated test runs on every push
- Add database-level verification for a real (non-mock) backend
- Extend the framework to cover additional resources such as `/posts` and `/comments`
- Add parallel test execution support

5.3 Repository

The complete source code for this project is available at: https://github.com/ShivrajCodes/Wipro_Selenium_Automation/tree/master/2_Capstone_Project/api-automation-framework